

第41章 呼吸灯与 SPWM 波

本章讲解的实验工程目录为：固件库例程\呼吸灯与 SPWM 波：TIM—单色呼吸灯、全彩呼吸灯、输出 SPWM 波。

学习本章需要全彩 LED 灯章节为基础。

41.1 呼吸灯简介

呼吸灯，就是指灯光设备的亮度随着时间由暗到亮逐渐增强，再由亮到暗逐渐衰减，很有节奏感地一起一伏，就像是在呼吸一样，因而被广泛应用于手机、电脑等电子设备的指示灯中，冰冷的电子设备应用呼吸灯后，顿时增添了几分温暖。

41.2 呼吸灯与 PWM 控制原理

呼吸的特性是一种类似图 41-1 中的指数曲线过程，吸气是指数上升过程，呼气是指数下降过程，成年人吸气呼气整个过程持续约 3 秒。

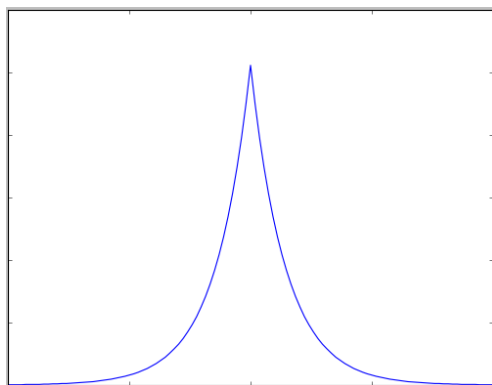


图 41-1 指数曲线

要控制 LED 灯达到呼吸灯的效果，实际上就是要控制 LED 灯的亮度拟合呼吸特性曲线。前面控制全彩 LED 灯时，通过控制脉冲的占空比来调整各个通道 LED 灯的亮度，从而达到混色的效果。若控制脉冲的占空比在 3 秒的时间周期内按呼吸特性曲线变化，那么就可以实现呼吸灯的效果了。

这种使用脉冲占空比拟合不同波形的方式称为 PWM(脉冲宽度调制)控制技术——通过对一系列脉冲的宽度进行调制，来等效地获得所需要波形（含形状和幅值）。PWM 控制的基本原理为：冲量相等而开头不同的窄脉冲加在具有惯性的环节上时，其效果基本相同。其中冲量指窄脉冲的面积；效果相同指环节输出响应波形基本相同。

例如：可以用一系列等幅不用一系列等幅不等宽的脉冲来代替一个正弦半波，见图 41-2。

- 把正弦半波 N 等分，可看成 N 个彼此相连的脉冲序列，宽度相等，但幅值不等；
- 用矩形脉冲代替，各个矩形脉冲等幅，不等宽，中点重合，脉冲宽度按正弦规律变化，脉冲的总面积（冲量）与正弦半波相等。

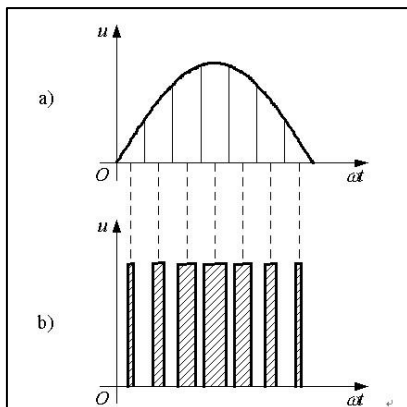


图 41-2 PWM 等效正弦波

这种脉冲波形被称为 SPWM 波形——脉冲宽度按正弦规律变化而和正弦波等效的 PWM 波形。SPWM 是一种非常典型的 PWM 波形，它在数字电路控制中应用非常广泛，如果使用低通滤波器，可以由 SPWM 波得到其等效的连续正弦半波。

要改变等效输出正弦波幅值，按同一比例改变各脉冲宽度即可。

若把拟合的波形改成呼吸特性曲线，即可得到控制呼吸灯使用的 PWM 波形，要生成拟合的 PWM 波形，通常使用算法法和调制法：

- (1) 算法法：根据拟合波形的频率、幅值和半周期脉冲数，准确计算 PWM 波各脉冲宽度和间隔，据此控制开关器件的通断，就可得到所需 PWM 波形；
- (2) 调制法：拟合波形作调制信号，进行调制得到期望的 PWM 波；该方法一般采用等腰三角波为载波，其任一点水平宽度和高度成线性关系且左右对称。载波（等腰三角波）与平缓变化的调制信号波（即要拟合的波形）相交，在载波与信号波的交点控制器件通断，就得宽度正比于信号波幅值的脉冲，符合 PWM 的要求，见图 41-3。相对于算法法，其处理过程计算简单。

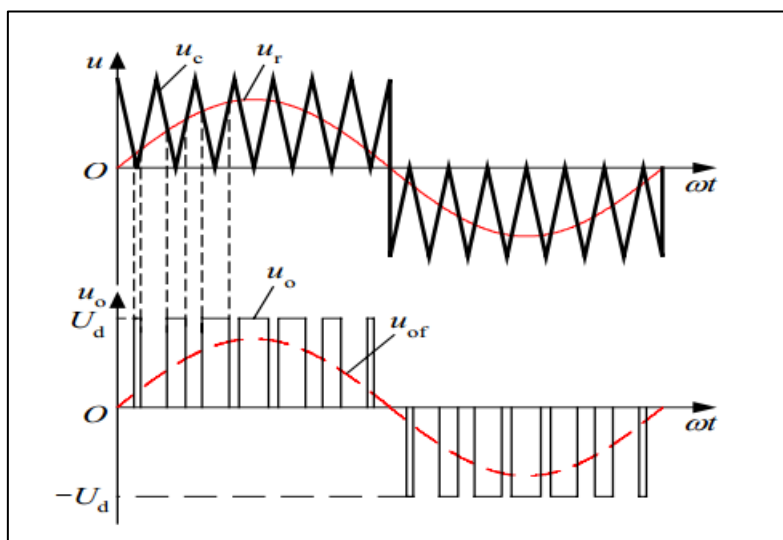


图 41-3 调制法得到 PWM 波

在本章的各个实验中，演示如何使用算法法得到的呼吸曲线 PWM 波和 SPWM 波，并使用 STM32 定时器 TIM 的 PWM 功能输出波形控制 LED 灯，达到呼吸灯的效果。

41.3 硬件设计

为便于讲解，本章设计了三个实验，其工程名为：TIM—单色呼吸灯、TIM—全彩呼吸灯和 TIM—输出 SPWM 波，它们的简要介绍见表 41-1。

表 41-1 各实验工程功能介绍

实验工程名称	说明
TIM—单色呼吸灯	使用呼吸曲线 PWM 波控制 LED 灯，可实现红、绿、蓝三种颜色的单色呼吸灯。
TIM—全彩呼吸灯	在单色呼吸灯的基础上，添加对 PWM 波的幅值控制，分为 256 个等级，从而实现 RGB888 全色彩的呼吸灯。
TIM—输出 SPWM 波	在全彩呼吸灯的基础上，把呼吸曲线 PWM 波改成 SPWM 波，演示使用 STM32 输出幅值可控的典型 SPWM 波。

本实验中使用的 RGB 灯硬件与前面全彩 LED 灯章节中的完全相同，见图 41-4，这是一个 RGB 灯，里面由红蓝绿三个小灯构成，使用 PWM 控制时可以混合成 256 不同的颜色。本实验与 LED 灯基础章节的主要差异是软件中的控制方式。

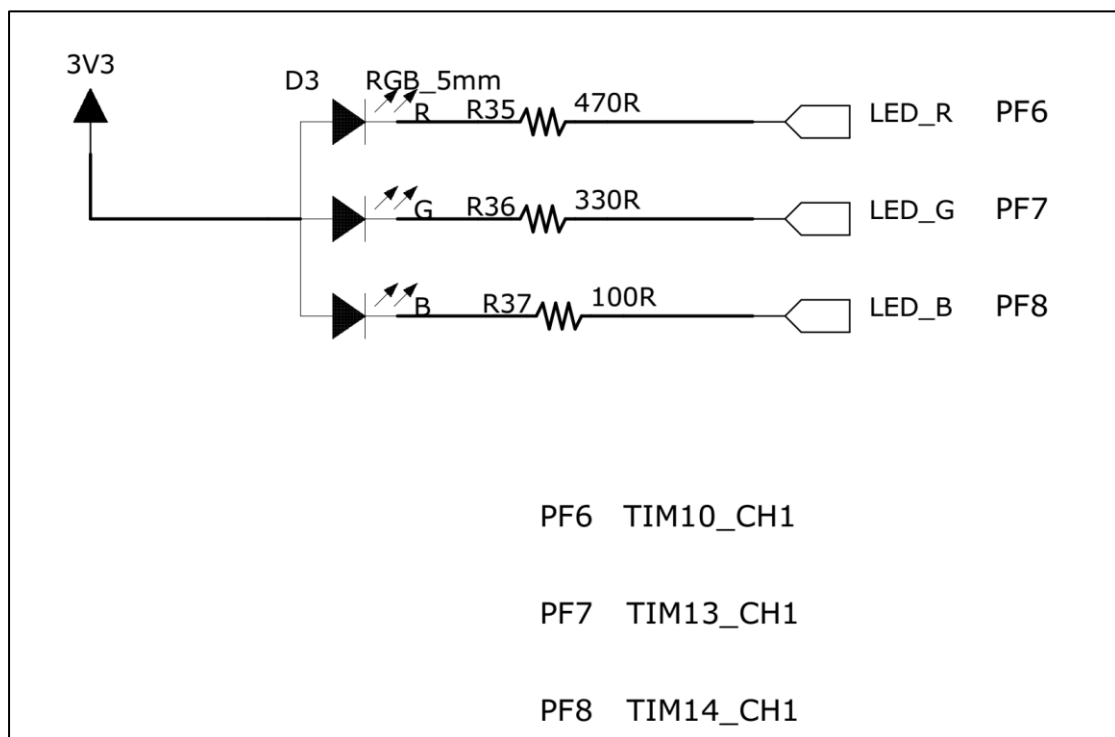


图 41-4 LED 硬件原理图

本开发板中设计的 RGB 灯控制引脚是经过仔细选择的，因为本实验的软件将使用 STM32 的定时器控制输出 PWM 脉冲，然而并不是任意 GPIO 都具有 STM32 定时器的输出通道功能，所以在设计硬件时，需要根据《STM32 数据手册》中的说明，选择具有定时器输出通道功能的引脚来控制 RGB 灯，见图 41-5。

Pin number						Pin name (function after reset) ⁽¹⁾	Pin type	I / O structure	Notes	Alternate functions	Additional functions
LQFP64	WLCSP90	LQFP100	LQFP144	UFBGA176	LQFP176						
-	-	-	18	K2	24	PF6	I/O	FT	(4)	TIM10_CH1 / FSMC_NIORD/ EVENTOUT	ADC3_IN4
-	-	-	19	K1	25	PF7	I/O	FT	(4)	TIM11_CH1/FSMC_NREG/ EVENTOUT	ADC3_IN5
-	-	-	20	L3	26	PF8	I/O	FT	(4)	TIM13_CH1 / FSMC_NIOWR/ EVENTOUT	ADC3_IN6

图 41-5 《STM32 中文数据手册》中本设计中使用的 LED 引脚说明

本实验中的 RGB 灯使用阴极分别连接到了 PF6、PF7 及 PF8，它们分别是定时器 TIM10、11、13 的通道 1。在本硬件设计方案中三个引脚使用了不同的定时器，它会给设计程序时会带来不便，若是在硬件资源分配充足的情况下，建议这三个控制引脚使用同一个定时器的不同通道。

41.4 单色呼吸灯实验

首先以单色呼吸灯工程为例，其核心的驱动代码分别位于 bsp_breath_led.c 和 bsp_breath_led.h 文件中，可根据应用需要移植这些文件。

41.4.1 编程要点

- (1) 初始化 PWM 输出通道，初始化 PWM 工作模式；
- (2) 计算获取 PWM 数据表；
- (3) 编写中断服务函数，在中断服务函数根据 PWM 数据表切换比较寄存器的值；

41.4.2 代码分析

1. LED 灯硬件相关宏定义

为方便迁移代码适应其它硬件设计，实验中把硬件相关的部分使用宏定义到 bsp_breath_led.h 文件中，使用不同硬件设计时，修改该文件即可，见代码清单 41-1。

代码清单 41-1 硬件相关宏定义（bsp_breath_led.h 文件）

```

1
2 #define RED_LIGHT      1
3 #define GREEN_LIGHT    2
4 #define BLUE_LIGHT     3

```

```
5
6 /*要使用什么颜色的呼吸灯，可选 RED_LIGHT、GREEN_LIGHT、BLUE_LIGHT*/
7 #define LIGHT_COLOR RED_LIGHT
8
9 /*****定时器通道*****/
10 #if LIGHT_COLOR == RED_LIGHT
11 //R 红色灯
12 #define BRE_TIM TIM10
13 #define BRE_TIM_CLK RCC_APB2Periph_TIM10
14 #define BRE_TIM_APBxClock_FUN RCC_APB2PeriphClockCmd
15
16 #define BRE_TIM_IRQn TIM1_UP_TIM10_IRQn
17 #define BRE_TIM_IRQHandler TIM1_UP_TIM10_IRQHandler
18
19 /*计算说明见 c 文件*/
20 /*部分通用定时器的时钟为 HCLK/4，部分为 HCLK/2*/
21 #define BRE_TIM_PRESCALER (470-1)
22
23
24 #define BRE_LED_PIN GPIO_Pin_6
25 #define BRE_LED_GPIO_PORT GPIOF
26 #define BRE_LED_GPIO_CLK RCC_AHB1Periph_GPIOF
27 #define BRE_LED_PINSOURCE GPIO_PinSource6
28 #define BRE_LED_AF GPIO_AF_TIM10
29
30 //通道比较寄存器，以 TIMx->CCRx 方式可访问该寄存器，设置新的比较值，控制占空比
31 //以宏封装后，使用这种形式：BRE_TIM->BRE_LED_CCRx，可访问该通道的比较寄存器
32 #define BRE_LED_CCRx CCR1
33 #define BRE_LED_TIM_CHANNEL TIM_Channel_1
34
35 #define BRE_TIM_OCxInit TIM_OC1Init //通道选择，1~4
36 #define BRE_TIM_OCxPreloadConfig TIM_OC1PreloadConfig
37
38
39 #elif LIGHT_COLOR == GREEN_LIGHT
40 //G 绿色灯
41 #define BRE_TIM TIM11
42 #define BRE_TIM_CLK RCC_APB2Periph_TIM11
43 #define BRE_TIM_APBxClock_FUN RCC_APB2PeriphClockCmd
44
45 #define BRE_TIM_IRQn TIM1_TRG_COM_TIM11_IRQn
46 #define BRE_TIM_IRQHandler TIM1_TRG_COM_TIM11_IRQHandler
47
48 /*计算说明见 c 文件*/
49 /*部分通用定时器的时钟为 HCLK/4，部分为 HCLK/2*/
50 #define BRE_TIM_PRESCALER (470-1)
51
52
53 #define BRE_LED_PIN GPIO_Pin_7
54 #define BRE_LED_GPIO_PORT GPIOF
55 #define BRE_LED_GPIO_CLK RCC_AHB1Periph_GPIOF
56 #define BRE_LED_PINSOURCE GPIO_PinSource7
57 #define BRE_LED_AF GPIO_AF_TIM11
58
59 //通道比较寄存器，以 TIMx->CCRx 方式可访问该寄存器，设置新的比较值，控制占空比
60 //以宏封装后，使用这种形式：BRE_TIM->BRE_LED_CCRx，可访问该通道的比较寄存器
61 #define BRE_LED_CCRx CCR1
62 #define BRE_LED_TIM_CHANNEL TIM_Channel_1
63
64 #define BRE_TIM_OCxInit TIM_OC1Init //通道选择，1~4
65 #define BRE_TIM_OCxPreloadConfig TIM_OC1PreloadConfig
66
67 #elif LIGHT_COLOR == BLUE_LIGHT
68 //B 蓝色灯
69 #define BRE_TIM TIM13
70 #define BRE_TIM_CLK RCC_APB1Periph_TIM13
```

```

71 #define BRE_TIM_APBxClock_FUN      RCC_APB1PeriphClockCmd
72
73 #define BRE_TIM_IRQn                TIM8_UP_TIM13_IRQn
74 #define BRE_TIM_IRQHandler          TIM8_UP_TIM13_IRQHandler
75
76 /*计算说明见 c 文件*/
77 /*部分通用定时器的时钟为 HCLK/4，部分为 HCLK/2*/
78 #define BRE_TIM_PRESCALER           (235-1)
79
80
81 #define BRE_LED_PIN                  GPIO_Pin_8
82 #define BRE_LED_GPIO_PORT            GPIOF
83 #define BRE_LED_GPIO_CLK             RCC_AHB1Periph_GPIOF
84 #define BRE_LED_PINSOURCE            GPIO_PinSource8
85 #define BRE_LED_AF                    GPIO_AF_TIM13
86
87 //通道比较寄存器，以 TIMx->CCRx 方式可访问该寄存器，设置新的比较值，控制占空比
88 //以宏封装后，使用这种形式：BRE_TIM->BRE_LED_CCRx，可访问该通道的比较寄存器
89 #define BRE_LED_CCRx                  CCR1
90 #define BRE_LED_TIM_CHANNEL           TIM_Channel_1
91
92 #define BRE_TIM_OCxInit                TIM_OC1Init      //通道选择，1~4
93 #define BRE_TIM_OCxPreloadConfig       TIM_OC1PreloadConfig
94
95 #endif

```

为方便切换 LED 灯的颜色，它定义了三组宏，通过修改代码中的“#define LIGHT_COLOR RED_LIGHT”语句，可以切换使用红、绿、蓝三种颜色的呼吸灯。

在每组宏定义中，与全彩 LED 灯实验中的类似，定义了定时器编号、定时器时钟使能库函数、引脚重映射操作、GPIO 端口和引脚号、通道对应的比较寄存器名以及中断通道和中断服务函数名。

与全彩 LED 灯实验不同，本实验中定时器的比较寄存器 CCRx 在控制呼吸灯的单个周期内需要切换为 PWM 表中不同的数值，所以需要利用定时器中断。

2. 初始化 GPIO

首先，初始化用于定时器输出通道的 GPIO，见代码清单 41-2。

代码清单 41-2 初始化 GPIO（bsp_breath_led.c 文件）

```

1 /**
2  * @brief 配置 TIM 复用输出 PWM 时用到的 I/O
3  * @param 无
4  * @retval 无
5  */
6 static void TIMx_GPIO_Config(void)
7 {
8     /*定义一个 GPIO_InitTypeDef 类型的结构体*/
9     GPIO_InitTypeDef GPIO_InitStructure;
10
11     /*开启 LED 相关的 GPIO 外设时钟*/
12     RCC_AHB1PeriphClockCmd ( BRE_LED_GPIO_CLK, ENABLE);
13
14     GPIO_PinAFConfig(BRE_LED_GPIO_PORT,BRE_LED_PINSOURCE,BRE_LED_AF);
15
16     /*BREATH_LED1*/
17     GPIO_InitStructure.GPIO_Pin = BRE_LED_PIN;
18     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF;
19     GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
20     GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL;

```

```
21     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_100MHz;  
22     GPIO_Init(BRE_LED_GPIO_PORT, &GPIO_InitStructure);  
23 }
```

由于本实验直接使用定时器输出通道的脉冲信号控制 LED 灯，此处代码把 GPIO 相关的引脚配置成了复用推挽输出模式，后面将使用定时器控制引脚进行 PWM 输出。

3. 定义 PWM 表

在本工程中的 bsp_breath_led.c 文件定义了一个 PWM 表，见代码清单 41-3。

代码清单 41-3 工程中定义的 PWM 表（bsp_breath_led.c 文件）

```
1  
2  /***** bsp_breath_led.c 文件*****/  
3  /* LED 亮度等级 PWM 表,指数曲线 ,  
4  此表使用工程目录下的 python 脚本 index_wave.py 生成*/  
5  uint16_t indexWave[] = {  
6      1, 1, 1, 1, 2, 2, 2, 2, 3, 3, 4, 4, 5, 5,  
7      6, 7, 8, 9, 10, 11, 13, 15, 17, 19, 22,  
8      25, 28, 32, 36, 41, 47, 53, 61, 69, 79,  
9      89, 102, 116, 131, 149, 170, 193, 219,  
10     250, 284, 323, 367, 417, 474, 539, 613,  
11     697, 792, 901, 1024, 1024, 901, 792, 697,  
12     613, 539, 474, 417, 367, 323, 284, 250,  
13     219, 193, 170, 149, 131, 116, 102, 89,  
14     79, 69, 61, 53, 47, 41, 36, 32, 28, 25,  
15     22, 19, 17, 15, 13, 11, 10, 9, 8, 7, 6,  
16     5, 5, 4, 4, 3, 3, 2, 2, 2, 2, 1, 1, 1, 1  
17 };  
18  
19 //计算 PWM 表有多少个元素  
20 uint16_t POINT_NUM = sizeof(indexWave)/sizeof(indexWave[0]);  
21
```

该 PWM 表是利用本工程目录下的 python 脚本 index_wave.py 生成的，该脚本的代码见代码清单 41-4。

代码清单 41-4 生成 PWM 表的 python 脚本

```
1  #! python3  
2  #coding=utf-8  
3  
4  """  
5  Python 版本: 3.x  
6  外部库: matplotlib1.5.3、numpy1.11.2  
7  
8  运行方式:  
9  在命令行中输入: python index_wave.py  
10  
11  运行结果:  
12  命令行中会打印计算得的各点数据,  
13  在当前目录下会生成 py_index_wave.c 文件, 包含上述数据,  
14  并且会弹出描绘曲线的对话框。  
15  """  
16  
17  import matplotlib.pyplot as plt  
18  import numpy as np  
19  import math  
20  
21  #修改本变量可以更改点数, 如 16、32、64 等  
22  POINT_NUM = 110  
23
```

```
24 #指数曲线最大为 2 的 MAX_POWER 次方
25 MAX_POWER = 10
26
27 # POINT_NUM 个点
28 x1 = np.linspace(0,MAX_POWER,POINT_NUM/2)
29
30 #f = 2^(x)
31 up=[]
32 for i in x1:
33     temp = round(2**i)
34     #得到升序列
35     up.append( temp )
36
37 x2 = np.linspace(MAX_POWER,2*MAX_POWER,POINT_NUM/2)
38
39 #f = 2^(2*MAX_POWER-x)
40 down=[]
41 for i in x2:
42     temp = round(2** (MAX_POWER*2-i))
43
44     #得到降序列
45     down.append( temp )
46
47 line = list(x1)+list(x2)
48 val = list(up)+list(down)
49
50 print(line)
51 print(" "*80)
52 print(list(map(int,val)))
53
54 #写入序列到文件
55 with open("py_index_Wave.c",'w',encoding= 'gb2312') as f:
56     print(list(map(int,val)),file= f)
57
58 #绘图
59 plt.plot(line,val,"-o")
60 plt.show()
61
62
63
```

该脚本运行后会在工程目录下生成一个包含该 PWM 表的 py_index_Wave.c 文件，复制这些数据到 bsp_breathing.c 文件，即可得到本工程中的 indexWave 数组。实际上使用 C 语言也可以编写这样的脚本制作出 PWM 表，只是使用 python 脚本比较方便绘制图形而已。

该 python 脚本生成 PWM 表数据的原理，实质是按照如下函数曲线进行采样：

若 $0 \leq x \leq 10$:

$$y = 2^x$$

若 $10 < x \leq 20$:

$$y = 2^{(20-x)}$$

python 脚本在这样的函数曲线上取 110 个点，即可得到上述代码中 PWM 表数组 indexWave，绘制其图形如图 41-6。

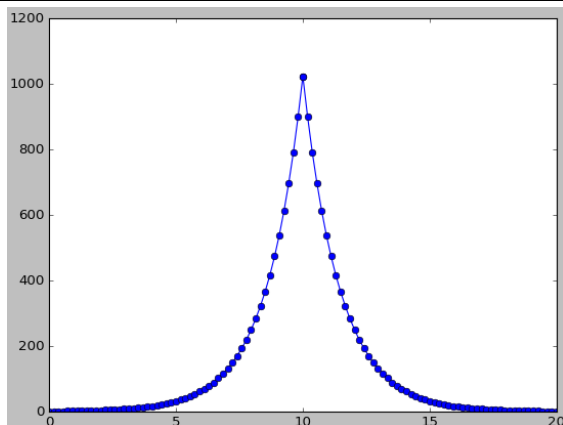


图 41-6 绘制 PWM 表的图形

可以看到，这个 PWM 表记录了呼吸特性曲线，在本实验中，PWM 表的数据将会被赋值到定时器的 CCRx 比较寄存器，从而控制输出占空比呈呼吸特性曲线变化的 PWM 波。

至于计算 PWM 表时，为什么选择采样 110 个点以及为什么表中的最大元素为 1024，需要结合下面定时器和中断服务函数中的配置来理解。

4. 定时器 PWM 配置

本实验的定时器的 PWM 输出模式配置，见代码清单 41-5。

代码清单 41-5 定时器 PWM 配置 (bsp_breath_led.c 文件)

```
1
2 /**
3  * @brief  通用定时器 TIMx, x[2,3,4,5] 中断优先级配置
4  * @param  无
5  * @retval 无
6  */
7 static void TIMx_NVIC_Configuration(void)
8 {
9     NVIC_InitTypeDef NVIC_InitStructure;
10     NVIC_PriorityGroupConfig(NVIC_PriorityGroup_1);
11
12     // 设置中断来源
13     NVIC_InitStructure.NVIC_IRQChannel = BRE_TIM_IRQn;
14
15     // 设置抢占优先级
16     NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 0;
17
18     // 设置子优先级
19     NVIC_InitStructure.NVIC_IRQChannelSubPriority = 2;
20     NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
21     NVIC_Init(&NVIC_InitStructure);
22 }
23
24
25
26 /**
27  * @brief  配置 TIM 输出的 PWM 信号的模式，如周期、极性
28  * @param  无
29  * @retval 无
30  */
31
32 static void TIM_Mode_Config(void)
33 {
```

```
34 TIM_TimeBaseInitTypeDef TIM_TimeBaseStructure;
35 TIM_OCInitTypeDef TIM_OCInitStructure;
36
37 // 开启 TIMx_CLK,x[2,3,4,5]
38 BRE_TIM_APBxClock_FUN(BRE_TIM_CLK, ENABLE);
39
40 /* 基本定时器配置 ,配合 PWM 表点数、中断服务函数中的 period_cnt 循环次数设置*/
41
42 /* 设置使得整个呼吸过程为 3 秒左右即可达到很好的效果 */
43
44 //要求:
45 //TIM_Period: 与 PWM 表中数值范围一致
46 //TIM_Prescaler: 越小越好,可减轻闪烁现象
47 //PERIOD_CLASS: 中断服务函数中控制单个点循环的次数,调整它可控制拟合曲线的周期
48 //POINT_NUM: PWM 表的元素,它是 PWM 拟合曲线的采样点数
49
50 /*****本实验中的配置*****/
51 /*****
52 #PWM 点数
53 POINT_NUM = 110
54
55 #周期倍数
56 PERIOD_CLASS = 10
57
58 #幅值等级
59 AMPLITUDE_CLASS = 1
60
61 #定时器定时周期
62 TIMER_TIM_Period = 2**10
63
64 #APB1 定时器分频
65 TIMER_TIM_Prescaler1 = 235
66 #APB2 定时器分频
67 TIMER_TIM_Prescaler2 = 470
68
69 #STM32 系统时钟频率和周期
70 f_hclk = 168000000
71
72 #定时器计时频率
73 f_pclk1 = f_hclk/2
74 t_pclk1 = 1/f_pclk1
75
76 f_pclk2 = f_hclk
77 t_pclk2 = 1/f_pclk2
78
79 #APB1 定时器 update 事件周期
80 t_timer1 = t_pclk1*TIMER_TIM_Prescaler1*TIMER_TIM_Period
81
82 #APB1 每个 PWM 点的时间
83 T_Point1 = t_timer1 * PERIOD_CLASS * AMPLITUDE_CLASS
84
85 #APB1 整个呼吸周期
86 T_Up_Down_Cycle1 = T_Point1 * POINT_NUM
87
88 print ("APB1 总线定时器呼吸周期: ",T_Up_Down_Cycle1)
89
90
91 #APB2 定时器 update 事件周期
92 t_timer2 = t_pclk2*TIMER_TIM_Prescaler2*TIMER_TIM_Period
93
94 #APB2 每个 PWM 点的时间
95 T_Point2 = t_timer2 * PERIOD_CLASS * AMPLITUDE_CLASS
96
97 #APB2 整个呼吸周期
```

```
98     T_Up_Down_Cycle2 = T_Point2 * POINT_NUM
99
100     print ("APB2 总线定时器呼吸周期: ",T_Up_Down_Cycle2)
101
102     #运行结果:
103
104     APB1 总线定时器呼吸周期: 3.151238
105     APB2 总线定时器呼吸周期: 3.151238
106     *****/
107     //当定时器从 0 计数到 TIM_Period+1 , 为一个定时周期
108     TIM_TimeBaseStructure.TIM_Period = (1024-1);
109
110     TIM_TimeBaseStructure.TIM_Prescaler = BRE_TIM_PRESCALER; //设置预分频
111     //设置时钟分频系数: 不分频(这里用不到)
112     TIM_TimeBaseStructure.TIM_ClockDivision = TIM_CKD_DIV1;
113     TIM_TimeBaseStructure.TIM_CounterMode = TIM_CounterMode_Up; //向上计数模式
114
115     // 初始化定时器 TIMx, x[2,3,4,5]
116     TIM_TimeBaseInit(BRE_TIM, &TIM_TimeBaseStructure);
117
118     /*PWM 模式配置*/
119     /* PWM1 Mode configuration: Channel1 */
120     TIM_OCInitStructure.TIM_OCMode = TIM_OCMode_PWM1; //配置为 PWM 模式
121     TIM_OCInitStructure.TIM_OutputState = TIM_OutputState_Enable; //使能输出
122     TIM_OCInitStructure.TIM_Pulse = 0; //设置初始 PWM 脉冲宽度为 0
123     //当定时器计数值小于 CCR1_Val 时为低电平, LED 灯亮
124     TIM_OCInitStructure.TIM_OCPolarity = TIM_OCPolarity_Low;
125
126     BRE_TIM_OCxInit(BRE_TIM, &TIM_OCInitStructure); //使能通道
127
128     /*使能通道重载*/
129     BRE_TIM_OCxPreloadConfig(BRE_TIM, TIM_OCPreload_Enable);
130
131     TIM_ARRPreloadConfig(BRE_TIM, ENABLE); //使能 TIM 重载寄存器 ARR
132
133     // 开启计数器中断
134     TIM_ITConfig(BRE_TIM, TIM_IT_Update, ENABLE);
135
136     // 使能计数器
137     TIM_Cmd(BRE_TIM, ENABLE);
138 }
```

本配置主体与全彩 LED 灯实验中的类似, 代码中初始化了控制 RGB 灯用的定时器, 它被配置为向上计数, PWM 通道输出也被配置成当计数器 CNT 的值小于输出比较寄存器 CCRx 的值时, PWM 通道输出低电平, 点亮 LED 灯。在函数的最后还使能了定时器中断, 每当定时器的一个计数周期完成时, 产生中断, 配合中断服务函数, 即可切换 CCRx 比较寄存器的值。

代码中的 TIM_Period 和 TIM_Prescaler 是关键配置。

其中 TIM_Period 被配置为(1024-1), 它控制控制定时器的定时周期, 定时器的计数寄存器 CNT 从 0 开始, 每个时钟会对计数器加 1, 计数至 1023 时完成一次计数, 产生中断, 也就是说一共 1024 个计数周期, 与 PWM 表元素中的最大值相同。若定时器的输出比较寄存器 CCRx 被赋值为 PWM 表中的元素, 即可改变输出对应占空比的 PWM 波, 控制 LED 灯, 如:

若 $CCR_x=1$ ，那么在 $CNT < CCR_x$ 时，通道输出低电平，LED 灯亮， $CNT > CCR_x$ 时，输出高电平，LED 灯灭，此时 $\frac{T_{LED_ON}}{T_{LED_OFF}} = \frac{1}{1024}$ ；

若 $CCR_x=474$ ，那么在 $CNT < CCR_x$ 时，通道输出低电平，LED 灯亮， $CNT > CCR_x$ 时，输出高电平，LED 灯灭，此时 $\frac{T_{LED_ON}}{T_{LED_OFF}} = \frac{474}{1024}$ ；

若 $CCR_x=1024$ ，那么在 $CNT < CCR_x$ 时，通道输出低电平，LED 灯亮， $CNT > CCR_x$ 时，输出高电平，LED 灯灭，此时 $\frac{T_{LED_ON}}{T_{LED_OFF}} = \frac{1024}{1024}$ 。

根据本工程中的 PWM 表更新 CCR_x 的值，即可输出占空比呈呼吸特性曲线变化的 PWM 波形，达到呼吸灯的效果。

最终，拟合曲线的周期由 TIMPeriod、PWM 表的点数、TIM_Prescaler 以及下面中断服务函数的 period_cnt 比较值共同决定，本工程需要调整这些参数使得拟合曲线的周期约为 3 秒，从而达到较平缓的呼吸效果。

5. 定时器中断服务函数

定时器的中断服务函数见代码清单 41-6。

代码清单 41-6 定时器中断服务函数 (stm32f4xx_it.c 文件)

```
1 //控制输出波形的频率
2 __IO uint16_t period_class = 10;
3
4 //计算 PWM 表有多少个元素
5 uint16_t POINT_NUM = sizeof(indexWave)/sizeof(indexWave[0]);
6
7 /**
8  * @brief This function handles TIM interrupt request.
9  * @param None
10  * @retval None
11  */
12 extern uint16_t indexWave[];
13
14 void BRE_TIM_IRQHandler (void)
15 {
16     static uint16_t pwm_index = 0;    //用于 PWM 查表
17     static uint16_t period_cnt = 0;   //用于计算周期数
18
19     if (TIM_GetITStatus(BRE_TIM, TIM_IT_Update) != RESET) { //TIM_IT_Update
20         period_cnt++;
21         //根据 PWM 表修改定时器的比较寄存器值
22         BRE_TIM->BRE_LED_CCRx = indexWave[pwm_index];
23
24         //每个 PWM 表中的每个元素使用 period_class 次
25         if (period_cnt > period_class) {
26             pwm_index++;    //标志 PWM 表指向下一个元素
27
28             //若 PWM 表已到达结尾，重新指向表头
29             if (pwm_index >= POINT_NUM) {
30                 pwm_index=0;
31             }
32
33             period_cnt=0;    //重置周期计数标志
```

```
34         } else {  
35         }  
36  
37         TIM_ClearITPendingBit (BRE_TIM, TIM_IT_Update); //必须要清除中断标志位  
38     }  
39 }
```

在中断服务函数中，包含两个静态变量 `period_cnt` 和 `pwm_index`。

其中 `pwm_index` 比较容易理解，它用于指示当前要使用 PWM 表中的哪个元素，从而在 “`BRE_TIMx->BRE_CCRx = indexWave[pwm_index];`” 语句中可以给 `CCRx` 赋予正确的数值，而且当 PWM 表中的数据都使用一遍时，`pwm_index` 将重新指向 PWM 表的开头，开始下一次呼吸循环。

在本实验的单次呼吸循环中，每个 PWM 表元素都会使用 10 次，代码中利用 `period_cnt` 变量指示当前使用的次数，当 `period_cnt > period_class` 时(即 `period_cnt > 10` 时)，`pwm_index` 才会指向下一个元素。每个 PWM 表元素使用多次，主要是为了在 `TIMPeriod`、PWM 表的点数、`TIM_Prescaler` 都固定的情况下，通过调整每个元素的重复次数可以调整整个拟合波形的周期。如把代码中的比较值 `period_class` 改为 100，每个 PWM 表遍历一次的时间就变为原来配置的 10 倍，其拟合的呼吸周期也就相应地改变了。图 41-7 说明了 `period = 3` 和 `period = 1` 时输出的 PWM 波形。

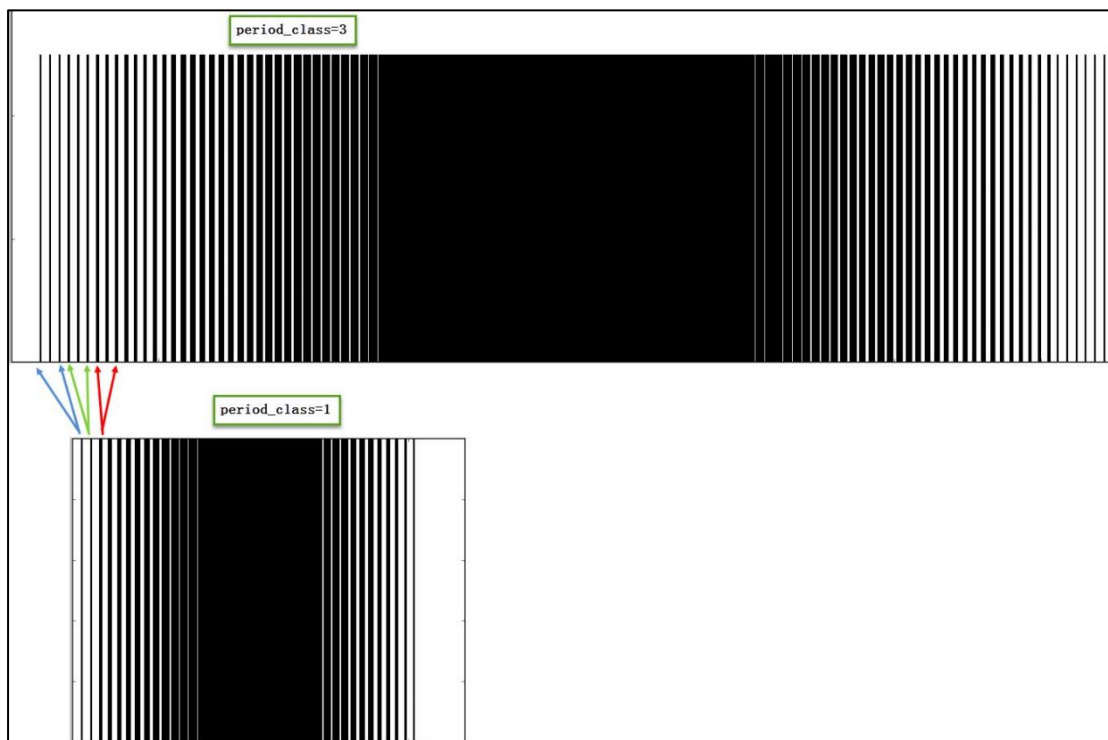


图 41-7 `period = 3` 和 `period = 1` 时输出的 PWM 波形

6. 计算拟合波形的周期

在本工程中，`TIMPeriod`、PWM 表的点数、`TIM_Prescaler` 以及 `period_cnt` 都会影响到拟合曲线的周期，而在实际应用中又有如下要求：

- **TIMPeriod**: 定时器的计数周期，它的值必须与 PWM 表中的极大值相等（应用中赋值需要减 1），而 PWM 表的极大值决定了控制的分辨率。例如极大值为 10 时，

PWM 占空比只有 10 个等级，精确到 0.1，当极大值为 1000 时，PWM 占空比有 1000 个等级，精确到 0.001。

- ❑ **TIM_Prescaler:** 定时器时钟分频因子，它控制定时器计数器 CNT 计数加 1 所需要的时间，它的值太大会导致输出的单个 PWM 波周期过长，影响控制的动态特性。如控制 LED 灯时，该值太大会导致 LED 灯开关时间变长，闪烁明显。一般来说，该值越小越好。
- ❑ **PWM 表的点数:** PWM 表的点数即对拟合曲线的采样点数，采样点越多，能更好地还原拟合曲线，采样点太少，可能会导致失真，见图 41-8。

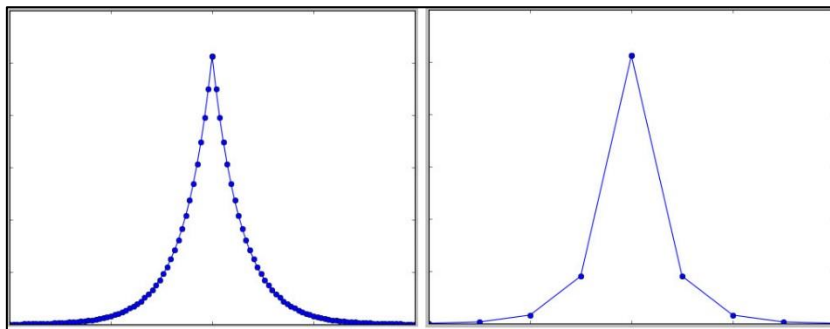


图 41-8 对呼吸特性曲线采样 110 个和 10 个点时的情况

- ❑ **period_class:** 周期倍数，即 PWM 表中每个元素的循环次数，它影响拟合曲线的周期。当 period_class=1 时，可以输出本配置中周期最短的拟合曲线。
- ❑ **amplitude_class:** 幅值分级，在后面全彩呼吸灯和 SPWM 实验中，我们还会增加该变量，它可以把拟合曲线的幅值分成 N 个等级，控制时可以选择按某个幅值等级进行输出。本实验中没有配置该参数，所以只能输出最大的等级，即 amplitude_class=1。

以上各个参数虽然侧重点不同，但若修改其中的任何一个，最终都会影响到所拟合曲线的周期，所以在实际应用中，通常先设定好 TIMPeriod、TIM_Prescaler、PWM 表的点数以及幅值等级数 amplitude_class，得到适合的控制精度、动态特性拟合度以及幅值等级后，然后再调整 period_class 控制拟合曲线的周期，而且 period_class 在程序中动态修改非常方便，不需要重置定时器和 PWM 表。

最终，我们把本工程配置中的拟合曲线周期计算公式概括如下：

(1) STM32 系统时钟默认频率和周期：

$$f_hclk = 168000000$$

某些定时器使用 APB1:

$$f_pclk1 = f_hclk/2$$

$$t_pclk1 = 1/f_pclk1$$

某些定时器使用 APB2:

$$f_pclk2 = f_hclk$$

$$t_pclk2 = 1/f_pclk2$$

(2) 定时器 update 事件周期，即定时器中断周期：

APB1 定时器: $t_timer1 = t_pclk1 * TIMER_TIM_Prescaler1 * TIMER_TIM_Period$

APB2 定时器: $t_timer2 = t_pclk2 * TIMER_TIM_Prescaler2 * TIMER_TIM_Period$

(3) 每个 PWM 点的时间：

APB1 定时器: $T_Point1 = t_timer1 * PERIOD_CLASS * AMPLITUDE_CLASS$

APB2 定时器: $T_Point2 = t_timer2 * PERIOD_CLASS * AMPLITUDE_CLASS$

(4) 最终，遍历 PWM 表的周期，即拟合曲线的周期：

APB1 定时器: $T_Up_Down_Cycle1 = T_Point1 * POINT_NUM$

APB2 定时器: $T_Up_Down_Cycle2 = T_Point2 * POINT_NUM$

例如，本工程配置中：

#PWM 点数

$POINT_NUM = 110$

#周期倍数

$PERIOD_CLASS = 10$

#幅值等级

$AMPLITUDE_CLASS = 1$

#定时器定时周期

$TIMER_TIM_Period = 1024$

#APB1 定时器分频

$TIMER_TIM_Prescaler1 = 235$

#APB2 定时器分频

$TIMER_TIM_Prescaler2 = 470$

#STM32 系统时钟频率和周期

$f_hclk = 168000000$

代入公式计算得：

APB1 总线定时器呼吸周期: $T_Up_Down_Cycle1 = 3.151238$

APB2 总线定时器呼吸周期: T_Up_Down_Cycle2 = 3.151238

通过公式的计算可知本工程的配置可使得输出的拟合曲线周期为 3.15 秒，是比较平缓的呼吸周期。

7. 主函数

代码清单 41-7 主函数

```
1 /**
2  * @brief 主函数
3  * @param 无
4  * @retval 无
5  */
6 int main(void)
7 {
8
9     /*初始化串口*/
10     Debug_USART_Config();
11
12     printf("\r\n 欢迎使用野火 STM32 F407 开发板.\r\n");
13     printf("\r\n 呼吸灯例程\r\n");
14     printf("\r\n RGB LED 以呼吸灯的形式闪烁\r\n ");
15
16     /* 初始化呼吸灯 */
17     BreathLED_Config();
18
19     while (1) {
20     }
21
22 }
```

main 函数中直接调用了 BreathLED_Config 函数，而该函数内部又直接调用了前面讲解的 GPIO 和 PWM 配置函数: TIMx_GPIO_Config 和 TIMx_Mode_Config。初始化完成后，定时器开始工作，然后它会在中断服务函数中切换 PWM 数据，控制 LED 灯显示呼吸效果。

41.4.3 下载验证

编译并下载本程序到开发板，给开发板上电复位，可看到 LED 灯显示呼吸效果。

41.5 全彩呼吸灯及输出 SPWM 波实验

全彩呼吸灯例程和输出 SPWM 波实验的工程基本一样，只是控制使用的 PWM 表不同，一个为呼吸特性曲线，另一个为正弦半波曲线，下面讲解主要以全彩呼吸灯实验为例。这两个工程的核心驱动代码分别位于 bsp_breath_led.c 和 bsp_breath_led.h 文件中，可根据应用需要移植这些文件。

这两个工程都是在单色呼吸灯例程拓展的，主要增加了对拟合曲线幅值等级的控制功能。

41.5.1 编程要点

- (1) 在单色呼吸灯的基础上，增加 PWM 输出通道，三个通道分别控制红绿蓝颜色；
- (2) 编写中断服务函数，增加对拟合波形幅值的控制；
- (3) 计算获取新的 PWM 数据表；

41.5.2 代码分析

下面主要以全彩呼吸灯的例程分析，输出 SPWM 波例程只是宏的名称和 PWM 表的数据不一样。

1. LED 灯硬件相关宏定义

类似地，实验中把硬件相关的部分使用宏定义到 bsp_breath_led.h 文件中，使用不同硬件设计时，修改该文件即可，见代码清单 41-8。

代码清单 41-8 硬件相关宏定义（bsp_breath_led.h 文件）

```
1
2 /*电压幅值等级数*/
3 #define AMPLITUDE_CLASS 256
4
5 //控制输出波形的频率
6 extern __IO uint16_t period_class ;
7
8 /*PWM 表中的点数*/
9 extern uint16_t POINT_NUM;
10
11 /*****定时器通道*****/
12
13 //R 红色灯
14 #define BRE_RED_TIM TIM10
15 #define BRE_RED_TIM_CLK RCC_APB2Periph_TIM10
16 #define BRE_TIM_GPIO_CLK (RCC_AHB1Periph_GPIOF)
17 #define BRE_RED_TIM_APBxClock_FUN RCC_APB2PeriphClockCmd
18
19 #define BRE_RED_TIM_IRQn TIM1_UP_TIM10_IRQn
20 #define BRE_RED_TIM_IRQHandler TIM1_UP_TIM10_IRQHandler
21
22 /*计算说明见 c 文件*/
23 /*部分通用定时器的时钟为 HCLK/4，部分为 HCLK/2*/
24 #define BRE_RED_TIM_PRESCALER (22-1)
25
26 #define BRE_RED_PIN GPIO_Pin_6
27 #define BRE_RED_GPIO_PORT GPIOF
28 #define BRE_RED_PINSOURCE GPIO_PinSource6
29 #define BRE_RED_AF GPIO_AF_TIM10
30
31 //通道比较寄存器，以 TIMx->CCRx 方式可访问该寄存器，设置新的比较值，控制占空比
32 //以宏封装后，使用这种形式：BRE_TIMx->BRE_LED_CCRx，可访问该通道的比较寄存器
33 #define BRE_RED_CCRx CCR1
34 #define BRE_RED_TIM_CHANNEL TIM_Channel_1
35
36 #define BRE_RED_TIM_OCxInit TIM_OC1Init //通道选择，1~4
```

```

37 #define BRE_RED_TIM_OCxPreloadConfig    TIM_OC1PreloadConfig
38
39 //G 绿色灯
40 #define BRE_GREEN_TIM                    TIM11
41 #define BRE_GREEN_TIM_CLK                RCC_APB2Periph_TIM11
42 #define BRE_GREEN_TIM_APBxClock_FUN      RCC_APB2PeriphClockCmd
43
44 #define BRE_GREEN_TIM_IRQn               TIM1_TRG_COM_TIM11_IRQn
45 #define BRE_GREEN_TIM_IRQHandler         TIM1_TRG_COM_TIM11_IRQHandler
46
47 /*计算说明见 c 文件*/
48 /*部分通用定时器的时钟为 HCLK/4，部分为 HCLK/2*/
49 #define BRE_GREEN_TIM_PRESCALER          (22-1)
50
51 #define BRE_GREEN_PIN                     GPIO_Pin_7
52 #define BRE_GREEN_GPIO_PORT              GPIOF
53 #define BRE_GREEN_PINSOURCE              GPIO_PinSource7
54 #define BRE_GREEN_AF                     GPIO_AF_TIM11
55
56 //通道比较寄存器，以 TIMx->CCRx 方式可访问该寄存器，设置新的比较值，控制占空比
57 //以宏封装后，使用这种形式：BRE_TIM->BRE_LED_CCRx，可访问该通道的比较寄存器
58 #define BRE_GREEN_CCRx                    CCR1
59 #define BRE_GREEN_TIM_CHANNEL             TIM_Channel_1
60
61 #define BRE_GREEN_TIM_OCxInit             TIM_OC1Init           //通道选择，1~4
62 #define BRE_GREEN_TIM_OCxPreloadConfig    TIM_OC1PreloadConfig
63
64 //B 蓝色灯
65 #define BRE_BLUE_TIM                     TIM13
66 #define BRE_BLUE_TIM_CLK                 RCC_APB1Periph_TIM13
67 #define BRE_BLUE_TIM_APBxClock_FUN       RCC_APB1PeriphClockCmd
68
69 #define BRE_BLUE_TIM_IRQn                TIM8_UP_TIM13_IRQn
70 #define BRE_BLUE_TIM_IRQHandler          TIM8_UP_TIM13_IRQHandler
71
72 /*计算说明见 c 文件*/
73 /*部分通用定时器的时钟为 HCLK/4，部分为 HCLK/2*/
74 #define BRE_BLUE_TIM_PRESCALER           (11-1)
75
76 #define BRE_BLUE_PIN                     GPIO_Pin_8
77 #define BRE_BLUE_GPIO_PORT              GPIOF
78 #define BRE_BLUE_PINSOURCE              GPIO_PinSource8
79 #define BRE_BLUE_AF                     GPIO_AF_TIM13
80
81 //通道比较寄存器，以 TIMx->CCRx 方式可访问该寄存器，设置新的比较值，控制占空比
82 //以宏封装后，使用这种形式：BRE_TIM->BRE_LED_CCRx，可访问该通道的比较寄存器
83 #define BRE_BLUE_CCRx                    CCR1
84 #define BRE_BLUE_TIM_CHANNEL             TIM_Channel_1
85
86 #define BRE_BLUE_TIM_OCxInit             TIM_OC1Init           //通道选择，1~4
87 #define BRE_BLUE_TIM_OCxPreloadConfig    TIM_OC1PreloadConfig

```

与单色呼吸灯的定义不同，本实验需要同时定义三组 PWM 通道，而且在代码的开头还增加了 AMPLITUDE_CLASS 宏，它在后面会被用于控制拟合曲线的电压等级，此处把它定义为 256，与 RGB 各通道的颜色等级一致，通过控制各通道的电压幅值，达到混合不同色彩的效果。

2. 初始化 GPIO

首先，初始化用于定时器输出通道的 GPIO，见代码清单 41-9。

代码清单 41-9 初始化 GPIO（bsp_breath_led.c 文件）

```

1 /**
2  * @brief 配置 TIM 复用输出 PWM 时用到的 I/O
3  * @param 无
4  * @retval 无
5  */
6 static void TIMx_GPIO_Config(void)
7 {
8     /*定义一个 GPIO_InitTypeDef 类型的结构体*/
9     GPIO_InitTypeDef GPIO_InitStructure;
10
11     /*开启 LED 相关的 GPIO 外设时钟*/
12     RCC_AHB1PeriphClockCmd ( BRE_TIM_GPIO_CLK, ENABLE);
13
14     GPIO_PinAFConfig(BRE_RED_GPIO_PORT,BRE_RED_PINSOURCE,BRE_RED_AF);
15     GPIO_PinAFConfig(BRE_GREEN_GPIO_PORT,BRE_GREEN_PINSOURCE,BRE_GREEN_AF);
16     GPIO_PinAFConfig(BRE_BLUE_GPIO_PORT,BRE_BLUE_PINSOURCE,BRE_BLUE_AF);
17
18     /* 配置 LED 灯用到的引脚 */
19     //红
20     GPIO_InitStructure.GPIO_Pin = BRE_RED_PIN;
21     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF;
22     GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
23     GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL;
24     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_100MHz;
25     GPIO_Init(BRE_RED_GPIO_PORT, &GPIO_InitStructure);
26
27     //绿
28     GPIO_InitStructure.GPIO_Pin = BRE_GREEN_PIN;
29     GPIO_Init(BRE_GREEN_GPIO_PORT, &GPIO_InitStructure);
30
31     //蓝
32     GPIO_InitStructure.GPIO_Pin = BRE_BLUE_PIN;
33     GPIO_Init(BRE_BLUE_GPIO_PORT, &GPIO_InitStructure);
34
35 }

```

本实验对 GPIO 的初始化也相对单色呼吸灯实验作了修改，同时初始化三个通道。

3. 定义 PWM 表

在全彩呼吸灯和 SPWM 波工程中定义了见代码清单 41-10 中的 PWM 表。

代码清单 41-10 工程中定义的 PWM 表（bsp_breathing.c 和 bsp_spwm.c 文件）

```

1
2 /*****全彩呼吸灯实验的指数曲线表*****/
3 /* LED 亮度等级 PWM 表,指数曲线 ,
4 此表使用工程目录下的 python 脚本 index_wave.py 生成*/
5 const uint16_t indexWave[] = {
6     1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2,
7     2, 2, 3, 3, 3, 3, 4, 4, 4, 4, 5,
8     5, 5, 6, 6, 7, 7, 8, 8, 9, 9, 10,
9     11, 12, 12, 13, 14, 15, 17, 18,
10    19, 20, 22, 23, 25, 27, 29, 31,
11    33, 36, 38, 41, 44, 47, 51, 54,
12    58, 63, 67, 72, 77, 83, 89, 95,
13    102, 110, 117, 126, 135, 145, 156,
14    167, 179, 192, 206, 221, 237, 254,
15    272, 292, 313, 336, 361, 387, 415,
16    445, 477, 512, 512, 477, 445, 415,
17    387, 361, 336, 313, 292, 272, 254,
18    237, 221, 206, 192, 179, 167, 156,
19    145, 135, 126, 117, 110, 102, 95,
20    89, 83, 77, 72, 67, 63, 58, 54, 51,

```

```
21    47, 44, 41, 38, 36, 33, 31, 29, 27,  
22    25, 23, 22, 20, 19, 18, 17, 15, 14,  
23    13, 12, 12, 11, 10, 9, 9, 8, 8, 7, 7,  
24    6, 6, 5, 5, 5, 4, 4, 4, 4, 3, 3, 3,  
25    3, 2, 2, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1, 1, 1,  
26 };  
27  
28 /*****SPWM 实验的正弦曲线表*****/  
29 /* SPWM 表,正弦曲线,此表使用工程目录下的python 脚本 sin_wave.py 生成*/  
30 const uint16_t indexWave[] = {  
31    0, 9, 18, 27, 36, 45, 54, 63, 72, 81, 89, 98,  
32    107, 116, 125, 133, 142, 151, 159, 168, 176,  
33    184, 193, 201, 209, 218, 226, 234, 242, 249,  
34    257, 265, 273, 280, 288, 295, 302, 310, 317,  
35    324, 331, 337, 344, 351, 357, 364, 370, 376,  
36    382, 388, 394, 399, 405, 410, 416, 421, 426,  
37    431, 436, 440, 445, 449, 454, 458, 462, 465,  
38    469, 473, 476, 479, 482, 485, 488, 491, 493,  
39    496, 498, 500, 502, 503, 505, 506, 508, 509,  
40    510, 510, 511, 512, 512, 512, 512, 512, 512,  
41    511, 510, 510, 509, 508, 506, 505, 503, 502,  
42    500, 498, 496, 493, 491, 488, 485, 482, 479,  
43    476, 473, 469, 465, 462, 458, 454, 449, 445,  
44    440, 436, 431, 426, 421, 416, 410, 405, 399,  
45    394, 388, 382, 376, 370, 364, 357, 351, 344,  
46    337, 331, 324, 317, 310, 302, 295, 288, 280,  
47    273, 265, 257, 249, 242, 234, 226, 218, 209,  
48    201, 193, 184, 176, 168, 159, 151, 142, 133,  
49    125, 116, 107, 98, 89, 81, 72, 63, 54, 45, 36,  
50    27, 18, 9, 0  
51 };  
52  
53 //计算 PWM 表有多少个元素  
54 uint16_t POINT_NUM = sizeof(indexWave)/sizeof(indexWave[0]);  
55
```

代码中列出的 PWM 表内元素的最大值均为 512，元素个数均为 180，把两个表绘制成曲线如图 41-9。

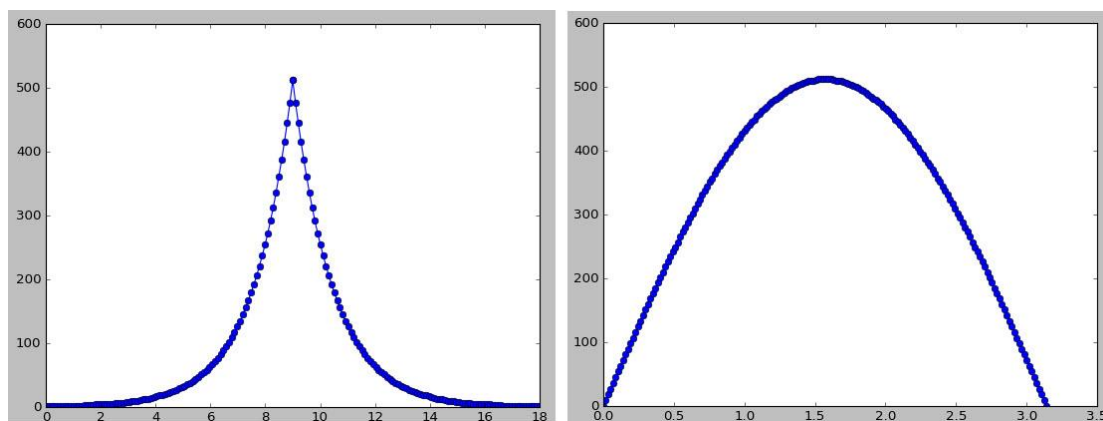


图 41-9 极值为 512，点数为 180 的呼吸曲线和正弦曲线

这些 PWM 表均由工程目录下的 python 脚本生成的，见代码清单 41-4 和代码清单 41-12。

代码清单 41-11 生成呼吸曲线 PWM 表的 python 脚本

```
1 #! python3  
2 #coding=utf-8  
3  
4 """  
5 Python 版本: 3.x
```

```
6 外部库: matplotlib1.5.3、numpy1.11.2
7
8 运行方式:
9 在命令行中输入: python index_wave.py
10
11 运行结果:
12 命令行中会打印计算得的各点数据,
13 在当前目录下会生成 py_index_wave.c 文件, 包含上述数据,
14 并且会弹出描绘曲线的对话框。
15 """
16
17 import matplotlib.pyplot as plt
18 import numpy as np
19 import math
20
21 #修改本变量可以更改点数, 如 16、32、64 等
22 POINT_NUM = 180
23
24 #指数曲线最大为 2 的 MAX_POWER 次方
25 MAX_POWER = 9
26
27 # POINT_NUM 个点
28 x1 = np.linspace(0, MAX_POWER, POINT_NUM/2)
29
30 #f = 2^(x)
31 up = []
32 for i in x1:
33     temp = round(2**i)
34     #得到升序列
35     up.append( temp )
36
37 x2 = np.linspace(MAX_POWER, 2*MAX_POWER, POINT_NUM/2)
38
39 #f = 2^(2*MAX_POWER-x)
40 down=[]
41 for i in x2:
42     temp = round(2**(MAX_POWER*2-i))
43
44     #得到降序列
45     down.append( temp )
46
47
48 line = list(x1)+list(x2)
49 val = list(up)+list(down)
50
51 print(line)
52 print("***80)
53 print(list(map(int, val)))
54
55 #写入序列到文件
56 with open("py_index_Wave.c", 'w', encoding= 'gb2312') as f:
57     print(list(map(int, val)), file= f)
58
59 #绘图
60 plt.plot(line, val, "-o")
61 plt.show()
62
```

以上脚本使用的采样函数如下:

若 $0 \leq x \leq 9$:

$$y = 2^x$$

若 $9 < x \leq 18$:

$$y = 2^{(18-x)}$$

该函数与单色呼吸灯实验中的类似，只是 x 的取值范围不同而导致 PWM 表的极大值不同。脚本运行后会在工程目录下生成一个包含该 PWM 表的 `py_index_Wave.c` 文件，复制这些数据到 `bsp_breath_led.c` 文件，即可得到工程中呼吸灯的 `indexWave` 数组。

生成正弦曲线的 SPWM 表脚本见代码清单 41-12。

代码清单 41-12 生成正弦曲线 PWM 表的 python 脚本

```
1  #! python3
2  #coding=utf-8
3
4  """
5  Python 版本: 3.x
6  外部库: matplotlib1.5.3、numpy1.11.2
7
8  运行方式:
9  在命令行中输入: python sinWave.py
10
11  运行结果:
12  命令行中会打印计算得的各点数据,
13  在当前目录下会生成 py_dac_sinWav.c 文件, 包含上述数据,
14  并且会弹出描绘曲线的对话框。
15  """
16
17  import matplotlib.pyplot as plt
18  import numpy as np
19  import math
20
21  #修改本变量可以更改点数, 如 16、32、64 等
22  POINT_NUM = 180
23  #正弦函数放大倍数
24  MULTIPLE = 2**9
25
26  #正弦函数 x 取值范围
27  X_VALUE = math.pi
28
29  # POINT_NUM 个点
30  n = np.linspace(0,X_VALUE,POINT_NUM)
31
32  #计算 POINT_NUM 个点的正弦值
33  a = map(math.sin,n)
34
35  r =[]
36  for i in a:
37      temp = round(i*MULTIPLE)
38      #得到序列
39      r.append( temp )
40
41  print(list(map(int,r)))
42
43  #写入序列到文件
44  with open("py_pwm_sinWave.c",'w',encoding= 'gb2312') as f:
45      print(list(map(int,r)),file= f)
46
47  #绘图
48  plt.plot(n,r,"-o")
49  plt.show()
```

以上生成 SPWM 的采样曲线函数如下:

$0 \leq x \leq \pi$:

$$y = 512 * \sin(x)$$

脚本运行后会在工程目录下生成一个包含该 PWM 表的 py_pwm_sinWave.c 文件，复制这些数据到 bsp_spwm.c 文件，即可得到工程中 SPWM 的 indexWave 数组。

使用这样的函数曲线，使得呼吸 PWM 表和 SPWM 表的极大值均为 512，而脚本中又控制两个表的采样点均为 180 个，在实际应用中可根据需求作不同的更改，此处仅为了演示方便，使用同样的配置。

4. 定时器 PWM 配置

本实验的定时器的 PWM 输出模式配置，见代码清单 41-13。

代码清单 41-13 定时器 PWM 配置（bsp_breathing.c 文件）

```
1 /**
2  * @brief  通用定时器 TIMx, x[2,3,4,5] 中断优先级配置
3  * @param  无
4  * @retval 无
5  */
6 static void TIMx_NVIC_Configuration(void)
7 {
8     NVIC_InitTypeDef NVIC_InitStructure;
9     NVIC_PriorityGroupConfig(NVIC_PriorityGroup_1);
10
11     //红
12     // 设置中断来源
13     NVIC_InitStructure.NVIC_IRQChannel = BRE_RED_TIM_IRQn;
14
15     // 设置抢占优先级
16     NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 0;
17
18     // 设置子优先级
19     NVIC_InitStructure.NVIC_IRQChannelSubPriority = 2;
20     NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
21     NVIC_Init(&NVIC_InitStructure);
22
23     //绿
24     // 设置中断来源
25     NVIC_InitStructure.NVIC_IRQChannel = BRE_GREEN_TIM_IRQn;
26     NVIC_Init(&NVIC_InitStructure);
27
28     //蓝
29     // 设置中断来源
30     NVIC_InitStructure.NVIC_IRQChannel = BRE_BLUE_TIM_IRQn;
31     NVIC_Init(&NVIC_InitStructure);
32
33 }
34
35
36 /**
37  * @brief  配置 TIM 输出的 PWM 信号的模式，如周期、极性
38  * @param  无
39  * @retval 无
40  */
41
42 static void TIM_Mode_Config(void)
43 {
44     TIM_TimeBaseInitTypeDef TIM_TimeBaseStructure;
45     TIM_OCInitTypeDef TIM_OCInitStructure;
46
```

```
47 // 开启 TIMx_CLK,x[2,3,4,5]
48 BRE_RED_TIM_APBxClock_FUN(BRE_RED_TIM_CLK,ENABLE);
49 BRE_GREEN_TIM_APBxClock_FUN(BRE_GREEN_TIM_CLK,ENABLE);
50 BRE_BLUE_TIM_APBxClock_FUN(BRE_BLUE_TIM_CLK,ENABLE);
51
52
53 /* 基本定时器配置 ,配合 PWM 表点数、中断服务函数中的 period_cnt 循环次数设置*/
54
55 /* 设置使得整个呼吸过程为 3 秒左右即可达到很好的效果 */
56
57 //要求:
58 //TIM_Period: 与 PWM 表中数值范围一致
59 //TIM_Prescaler: 越小越好,可减轻闪烁现象
60 //PERIOD_CLASS: 中断服务函数中控制单个点循环的次数,调整它可控制拟合曲线的周期
61 //POINT_NUM: PWM 表的元素,它是 PWM 拟合曲线的采样点数
62
63 /*****本实验中的配置*****/
64 /*****
65 #PWM 点数
66 POINT_NUM = 180
67
68 #周期倍数
69 PERIOD_CLASS = 1
70
71 #幅值等级
72 AMPLITUDE_CLASS = 256
73
74 #定时器定时周期
75 TIMER_TIM_Period = 2**9
76
77 #APB1 定时器分频
78 TIMER_TIM_Prescaler1 = 11
79 #APB2 定时器分频
80 TIMER_TIM_Prescaler2 = 22
81
82 #STM32 系统时钟频率和周期
83 f_hclk = 168000000
84
85 #定时器计时频率
86 f_pclk1 = f_hclk/2
87 t_pclk1 = 1/f_pclk1
88
89 f_pclk2 = f_hclk
90 t_pclk2 = 1/f_pclk2
91
92 #APB1 定时器 update 事件周期
93 t_timer1 = t_pclk1*TIMER_TIM_Prescaler1*TIMER_TIM_Period
94
95 #APB1 每个 PWM 点的时间
96 T_Point1 = t_timer1 * PERIOD_CLASS * AMPLITUDE_CLASS
97
98 #APB1 整个呼吸周期
99 T_Up_Down_Cycle1 = T_Point1 * POINT_NUM
100
101 print ("APB1 总线定时器呼吸周期: ",T_Up_Down_Cycle1)
102
103
104 #APB2 定时器 update 事件周期
105 t_timer2 = t_pclk2*TIMER_TIM_Prescaler2*TIMER_TIM_Period
106
107 #APB2 每个 PWM 点的时间
108 T_Point2 = t_timer2 * PERIOD_CLASS * AMPLITUDE_CLASS
109
110 #APB2 整个呼吸周期
```

```
111     T_Up_Down_Cycle2 = T_Point2 * POINT_NUM
112
113     print ("APB2 总线定时器呼吸周期: ", T_Up_Down_Cycle2)
114
115
116
117     #运行结果:
118
119     APB1 总线定时器呼吸周期: 3.08955
120     APB2 总线定时器呼吸周期: 3.08955
121     *****/
122     //当定时器从 0 计数到 255, 即为 256 次, 为一个定时周期
123     TIM_TimeBaseStructure.TIM_Period = 512-1;
124     TIM_TimeBaseStructure.TIM_Prescaler = BRE_RED_TIM_PRESCALER;
125     //设置时钟分频系数: 不分频(这里用不到)
126     TIM_TimeBaseStructure.TIM_ClockDivision = TIM_CKD_DIV1;
127     TIM_TimeBaseStructure.TIM_CounterMode = TIM_CounterMode_Up; //向上计数模式
128
129     // 初始化定时器 TIMx
130     TIM_TimeBaseInit(BRE_RED_TIM, &TIM_TimeBaseStructure);
131
132
133     /*基本定时器配置 TIM_Prescaler 根据效果来设置即可, 中断周期小
134     灯闪烁快, 大则闪烁缓慢*/
135     TIM_TimeBaseStructure.TIM_Prescaler = BRE_GREEN_TIM_PRESCALER;
136     // 初始化定时器 TIMx
137     TIM_TimeBaseInit(BRE_GREEN_TIM, &TIM_TimeBaseStructure);
138
139     /*基本定时器配置 TIM_Prescaler 根据效果来设置即可, 中断周期小
140     灯闪烁快, 大则闪烁缓慢*/
141     TIM_TimeBaseStructure.TIM_Prescaler = BRE_BLUE_TIM_PRESCALER;
142     // 初始化定时器 TIMx
143     TIM_TimeBaseInit(BRE_BLUE_TIM, &TIM_TimeBaseStructure);
144
145     /*PWM 模式配置*/
146     /* PWM1 Mode configuration: Channel1 */
147     TIM_OCInitStructure.TIM_OCMode = TIM_OCMode_PWM1; //配置为 PWM 模式
148
149     TIM_OCInitStructure.TIM_OutputState = TIM_OutputState_Enable; //使能输出
150     TIM_OCInitStructure.TIM_Pulse = 0; //设置初始 PWM 脉冲宽度为 0
151     //当定时器计数值小于 CCR_Val 时为低电平 LED 灯亮
152     TIM_OCInitStructure.TIM_OCPolarity = TIM_OCPolarity_Low;
153
154     //使能通道
155     BRE_RED_TIM_OCxInit(BRE_RED_TIM, &TIM_OCInitStructure);
156     /*使能通道重载*/
157     BRE_RED_TIM_OCxPreloadConfig(BRE_RED_TIM, TIM_OCPreload_Enable);
158
159     //使能通道
160     BRE_GREEN_TIM_OCxInit(BRE_GREEN_TIM, &TIM_OCInitStructure);
161     /*使能通道重载*/
162     BRE_GREEN_TIM_OCxPreloadConfig(BRE_GREEN_TIM, TIM_OCPreload_Enable);
163
164     //使能通道
165     BRE_BLUE_TIM_OCxInit(BRE_BLUE_TIM, &TIM_OCInitStructure);
166     /*使能通道重载*/
167     BRE_BLUE_TIM_OCxPreloadConfig(BRE_BLUE_TIM, TIM_OCPreload_Enable);
168
169     //使能 TIM 重载寄存器 ARR
170     TIM_ARRPreloadConfig(BRE_RED_TIM, ENABLE);
171     TIM_ARRPreloadConfig(BRE_GREEN_TIM, ENABLE);
172     TIM_ARRPreloadConfig(BRE_BLUE_TIM, ENABLE);
173
174     // 开启计数器中断
```

```
174 TIM_ITConfig(BRE_RED_TIM, TIM_IT_Update, ENABLE);
175 TIM_ITConfig(BRE_GREEN_TIM, TIM_IT_Update, ENABLE);
176 TIM_ITConfig(BRE_BLUE_TIM, TIM_IT_Update, ENABLE);
177
178 // 使能计数器
179 TIM_Cmd(BRE_RED_TIM, ENABLE);
180 TIM_Cmd(BRE_GREEN_TIM, ENABLE);
181 TIM_Cmd(BRE_BLUE_TIM, ENABLE);
182 }
```

本配置中同时使能三个 PWM 通道，而定时器初始化中的 TIM_Period 成员被配置为 (512-1)，即 PWM 表元素的极大值，TIM_Prescaler 被配置为 11 和 22，相对于单色呼吸灯实验，提高了定时器的时钟频率。

5. 定时器中断服务函数

定时器的中断服务函数见代码清单 41-6。

代码清单 41-14 定时器中断服务函数（stm32f4xx_it.c 文件）

```
1
2 /*电压幅值等级数*/
3 #define AMPLITUDE_CLASS 256
4
5 //控制输出波形的频率
6 __IO uint16_t period_class = 1;
7
8 extern uint16_t indexWave[];
9
10 extern __IO uint32_t rgb_color;
11
12 void BRE_RED_TIM_IRQHandler (void)
13 {
14     static uint16_t pwm_index = 0;        //用于 PWM 查表
15     static uint16_t period_cnt = 0;       //用于计算周期数
16     static uint16_t amplitude_cnt = 0;    //用于计算幅值等级
17
18     if (TIM_GetITStatus(BRE_RED_TIM, TIM_IT_Update) != RESET) { //TIM_IT_Update
19         amplitude_cnt++;
20
21         //每个 PWM 表中的每个元素有 AMPLITUDE_CLASS 个等级，
22         //每增加一级多输出一次脉冲，即 PWM 表中的元素多使用一次
23         //使用 256 次，根据 RGB 颜色分量设置通道输出
24         if (amplitude_cnt > (AMPLITUDE_CLASS-1)) {
25             period_cnt++;
26
27             //每个 PWM 表中的每个元素使用 period_class 次
28             if (period_cnt > period_class) {
29                 pwm_index++;        //标志 PWM 表指向下一个元素
30
31                 //若 PWM 表已到达结尾，重新指向表头
32                 if (pwm_index >= POINT_NUM) {
33                     pwm_index=0;
34                 }
35                 period_cnt = 0;      //重置周期计数标志
36             }
37             amplitude_cnt=0;        //重置幅值计数标志
38         } else {
39             //每个 PWM 表中的每个元素有 AMPLITUDE_CLASS 个等级，
40             //每增加一级多输出一次脉冲，即 PWM 表中的元素多使用一次
41
42             //根据 RGB 颜色分量值，设置各个通道是否输出当前的 PWM 表元素表示的亮度
```

```
43         //红
44         if (((rgb_color&0xFF0000)>>16) >= amplitude_cnt)
45             //根据 PWM 表修改定时器的比较寄存器值
46             BRE_RED_TIM->BRE_RED_CCRx = indexWave[pwm_index];
47         else
48             //比较寄存器值为 0，通道输出高电平，该通道 LED 灯灭
49             BRE_RED_TIM->BRE_RED_CCRx = 0;
50     }
51     TIM_ClearITPendingBit (BRE_RED_TIM, TIM_IT_Update); //必须要清除中断标志位
52 }
53 }
54 /*绿灯和蓝灯的中断服务函数类似，此处省略...*/
```

本中断服务函数相对于单色呼吸灯例程增加了对拟合曲线电压等级的控制，它是利用计数变量 `amplitude_cnt`、电压分级宏 `AMPLITUDE_CLASS` 以及电压等级变量 `rgb_color` 实现的，其实现原理如下：

为便于讲解，先假设用于配置拟合波形周期长度的 `period_class` 值为 1，即在控制周期长度时，每个 PWM 表中的元素只使用 1 次（关于 `period_class` 的作用请复习前面单色呼吸灯实验中的说明）。

在这个基础上增加对电压的分级，用于控制拟合波形的输出电压，本实验中由宏 `AMPLITUDE_CLASS` 控制，其值为 256，即可输出 256 种不同的电压等级。

该宏值在中断中会与 `amplitude_cnt` 进行比较（第 19 行），`amplitude_cnt` 每次进入中断加 1，当 `amplitude_cnt` 大于 `AMPLITUDE_CLASS` 时才会进入周期配置的判断，以便使 PWM 表指向下一个元素，也就是说增加电压分级配置后，遍历每个 PWM 表元素时，每个元素会增加 256 个周期。

在这 256 个周期内，会进入控制电压等级的处理（第 38~49 行），处理的过程是使用电压等级值 `rgb_color` 与 `amplitude_cnt` 进行比较，本实验中 `rgb_color` 包含红绿蓝三个通道的电压值，各通道的取值范围是[0:255]，与 RGB888 颜色格式一致。当通道的电压值 R/G/B 数据大于 `amplitude_cnt` 时，向该通道的比较寄存器赋予 PWM 表中当前指向的元素值，否则赋予 0 值。根据定时器的配置可知，比较寄存器中的值就是该通道输出低电平的时间，即 LED 灯亮的时间，所以，在 256 个周期时间内，各通道有 R/G/B 个周期会点亮 LED 灯当前 PWM 表元素表示的时间。

例如：

若红色通道值 `R=0`，那么 R 值在 256 个周期内，均小于 `amplitude_cnt`，通道输出高电平，红色 LED 灯一直灭，此时 $\frac{T_{R_LED_ON}}{T_{CLASS}} = \frac{0}{256}$ ；

若绿色通道值 `G=128`，那么 G 值在前 128 个周期内大于 `amplitude_cnt`，通道输出当前 PWM 表元素表示时间的低电平，绿色 LED 灯亮 128 个周期，此时 $\frac{T_{G_LED_ON}}{T_{CLASS}} = \frac{128}{256}$ ；

若蓝色通道值 `B=200`，那么 B 值在前 200 个周期内大于 `amplitude_cnt`，通道输出当前 PWM 表元素表示时间的低电平，蓝色 LED 灯亮 200 个周期，此时 $\frac{T_{B_LED_ON}}{T_{CLASS}} = \frac{200}{256}$ 。

所以，三个通道控制的 LED 灯点亮的时间比例即为 RGB888 颜色值表示的量，混合后可得到该颜色。

又由于 PWM 表中的元素值则表示了混合颜色的亮度，把 PWM 表遍历一遍，即控制混合颜色亮度呈 PWM 表变化，即可得到该颜色的呼吸灯效果。

当周期倍数 `period_class=1`，电压分级数量 `AMPLITUDE_CLASS=5`，电压等级分别为 3 和 1 时，输出的 PWM 波形如图 41-10 所示。

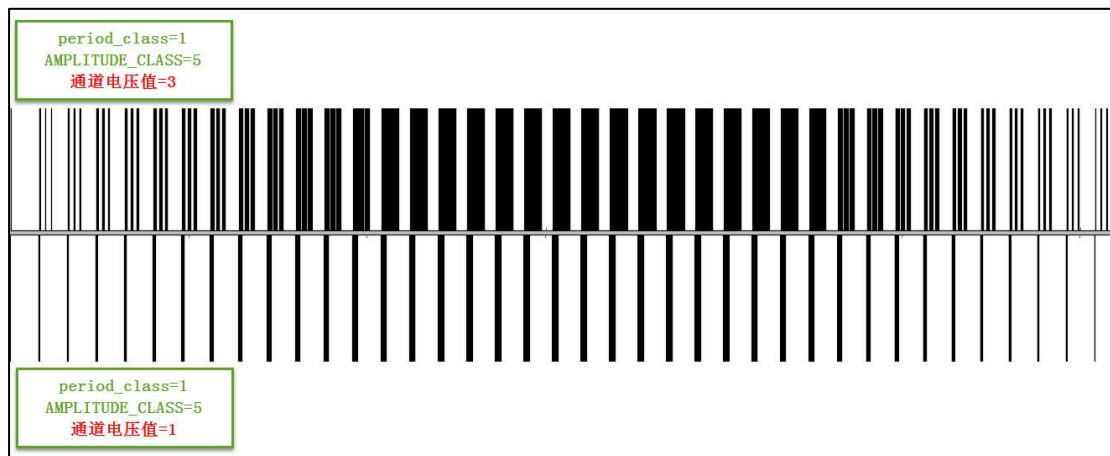


图 41-10 电压等级分别为 3 和 1 时的 PWM 波形

推广至当 `period_class` 不等于 1 时，遍历 PWM 表中的每个元素需要进入定时器中断 $AMPLITUDE_CLASS * period_class$ 次，设定定时器的中断周期为 T_timer ，那么输出的拟合曲线周期为 $T_timer * AMPLITUDE_CLASS * period_class$ 。

6. 计算拟合波形的周期

本工程相对单色呼吸灯例程增加了电压分级 `AMPLITUDE_CLASS`，结合前面的 `TIMPeriod`、PWM 表的点数、`TIM_Prescaler` 以及 `period_cnt` 参数，重新总结如下：

- ❑ **TIMPeriod:** 定时器的计数周期，它的值必须与 PWM 表中的极大值相等（应用中赋值需要减 1），而 PWM 表的极大值决定了控制的分辨率。例如极大值为 10 时，PWM 占空比只有 10 个等级，精确到 0.1，当极大值为 1000 时，PWM 占空比有 1000 个等级，精确到 0.001。
- ❑ **TIM_Prescaler:** 定时器时钟分频因子，它控制定时器计数器 CNT 计数加 1 所需要的时间，它的值太大会导致输出的单个 PWM 波周期过长，影响控制的动态特性。如控制 LED 灯时，该值太大会导致 LED 灯开关时间变长，闪烁明显。一般来说，该值越小越好。
- ❑ **PWM 表的点数:** PWM 表的点数即对拟合曲线的采样点数，采样点越多，能更好地还原拟合曲线，采样点太少，可能会导致失真，见图 41-8。

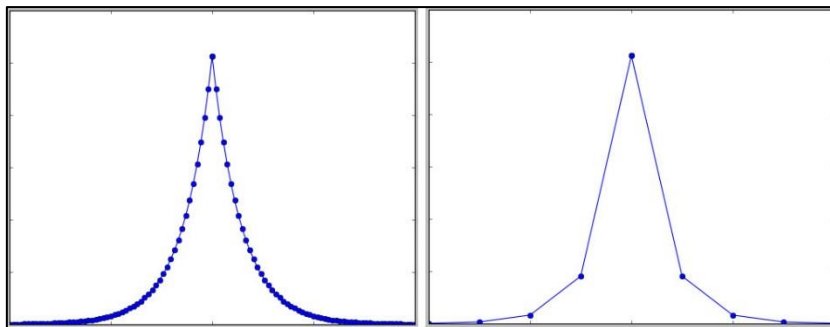


图 41-11 对呼吸特性曲线采样 110 个和 10 个点时的情况

- **period_class**: PWM 表中每个元素的循环次数，它影响拟合曲线的周期。当 **period_class=1** 时，可以输出本配置中周期最短的拟合曲线。
- **AMPLITUDE_CLASS**: 电压分级数，它可以把输出拟合曲线的幅值分成 **N** 个等级，控制时可以选择按某个幅值等级进行输出，可根据实际情况进行分级，如本实验中分级为与 RGB888 各通道一致的 256 等级，若只需要支持 RGB555 格式，那么 **AMPLITUDE_CLASS** 配置为 32 即可。

以上各个参数虽然侧重点不同，但若修改其中的任何一个，最终都会影响到所拟合曲线的周期，所以在实际应用中，通常先设定好 **TIMPeriod**、**TIM_Prescaler**、PWM 表的点数以及幅值等级数 **AMPLITUDE_CLASS**，得到适合的控制精度、动态特性拟合度以及幅值等级后，然后再调整 **period_class** 控制拟合曲线的周期，而且 **period_class** 在程序中动态修改非常方便，不需要重置定时器和 PWM 表。

最终，我们把本工程配置中的拟合曲线周期计算公式概括如下：

- (1) STM32 系统时钟默认频率和周期：

$$f_hclk = 168000000$$

某些定时器使用 APB1:

$$f_pclk1 = f_hclk/2$$

$$t_pclk1 = 1/f_pclk1$$

某些定时器使用 APB2:

$$f_pclk2 = f_hclk$$

$$t_pclk2 = 1/f_pclk2$$

- (2) 定时器 update 事件周期，即定时器中断周期：

APB1 定时器: $t_timer1 = t_pclk1 * \text{TIMER_TIM_Prescaler1} * \text{TIMER_TIM_Period}$

APB2 定时器: $t_timer2 = t_pclk2 * \text{TIMER_TIM_Prescaler2} * \text{TIMER_TIM_Period}$

- (3) 每个 PWM 点的时间：

APB1 定时器: $T_Point1 = t_timer1 * \text{PERIOD_CLASS} * \text{AMPLITUDE_CLASS}$

APB2 定时器: $T_Point2 = t_timer2 * \text{PERIOD_CLASS} * \text{AMPLITUDE_CLASS}$

(4) 最终，遍历 PWM 表的周期，即拟合曲线的周期：

APB1 定时器: $T_Up_Down_Cycle1 = T_Point1 * POINT_NUM$

APB2 定时器: $T_Up_Down_Cycle2 = T_Point2 * POINT_NUM$

例如，本工程配置中：

#PWM 点数

POINT_NUM = 180

#周期倍数

PERIOD_CLASS = 1

#幅值等级

AMPLITUDE_CLASS = 256

#定时器定时周期

TIMER_TIM_Period = 512

#APB1 定时器分频

TIMER_TIM_Prescaler1 = 11

#APB2 定时器分频

TIMER_TIM_Prescaler2 = 22

#STM32 系统时钟频率和周期

f_hclk = 168000000

代入公式计算得：

APB1 总线定时器呼吸周期: $T_Up_Down_Cycle1 = 3.08955$

APB2 总线定时器呼吸周期: $T_Up_Down_Cycle2 = 3.08955$

通过公式的计算可知本工程的配置可使得输出的拟合曲线周期为 3.08955 秒，是比较平缓的呼吸周期。

7. 主函数

代码清单 41-15 主函数

-
- 1 //该变量在定时器中断服务函数中使用，用于控制各通道的输出
 - 2 //修改该变量的值可直接改变呼吸灯的颜色

```
3 //变量格式: RGB888
4 __IO uint32_t rgb_color = 0xFF00FF;
5
6
7
8 /**
9  * @brief 主函数
10  * @param 无
11  * @retval 无
12  */
13 int main(void)
14 {
15     /* 初始化呼吸灯 */
16     BreathLED_Config();
17
18     while (1) {
19
20         // 可动态修改颜色, 使用各种颜色的呼吸灯
21         rgb_color = 0xFF00FF;
22         SOFT_DELAY();
23
24         rgb_color = 0x8080ff;
25         SOFT_DELAY();
26
27         rgb_color = 0xff8000;
28         SOFT_DELAY();
29
30         rgb_color = 0xffc90e;
31         SOFT_DELAY();
32
33     }
34 }
```

main 函数中初始化呼吸灯使用的定时器和 GPIO 后, 定时器开始工作, 然后它会在中断服务函数中切换 PWM 数据, 并且根据 rgb_color 的值控制拟合曲线的电压值, 达到控制 LED 灯可以各种颜色显示呼吸效果。在 main 函数中, 可通过修改 rgb_color 的值可以改变呼吸灯的颜色。

对于输出 SPWM 波的工程, 也直接使用了定时器输出的 SPWM 波控制 RGB 彩灯, 因为正弦曲线和呼吸曲线变化方向类似, 所以它控制 RGB 灯时, 看起来也有呼吸灯的效果, 在实际应用中可以使用类似的方法控制 SPWM 波拟合正弦曲线的频率和电压, 通过定时器 PWM 模式初始化中的结构体成员 TIM_OCInitStructure.TIM_OCPolarity 可控制当定时器计数值小于 CCRx_Val 时为高电平还是低电平。

41.5.3 下载验证

编译并下载本程序到开发板, 给开发板上电复位, 可看到 LED 灯显示呼吸效果。